

AI Applications Lecture 3

Functions Represented by Neural Networks

SUZUKI, Atsushi^a

^aI thank my collaborator, Jing WANG, for cooperating in creating the teaching materials.

Outline

1. Introduction
2. Preparation: Mathematical Notations
3. What does “neural network” truly refer to? - DAG-based parametric function
4. Does the DAG-based definition include practical neural networks?
5. Architecture and Checkpoints
6. Summary and Future Outlook

Review of the previous lecture

In the last lecture, we saw that many machine learning models, including neural networks, can be understood within a unified framework.

- Many machine learning models can be formulated as a **parametric function** $f_{(\cdot)}$, where the specific input-output relation f_{θ} is determined by the values of parameters θ .

Review of the previous lecture

In the last lecture, we saw that many machine learning models, including neural networks, can be understood within a unified framework.

- Many machine learning models can be formulated as a **parametric function** $f_{(\cdot)}$, where the specific input-output relation f_{θ} is determined by the values of parameters θ .
- The process of solving a task is divided into two phases.
 - **Training:** Using data to find the parameter θ^* that best solves the task.
 - **Inference:** Using the trained parameter θ^* to provide a new input to the function f_{θ^*} and obtain an output.

Learning outcomes of this lecture

Through this lecture, students will aim to be able to:

Learning outcomes of this lecture

Through this lecture, students will aim to be able to:

- Handle various neural networks (NNs) under the general definition of the NN as a **directed acyclic graph (DAG)**.

Learning outcomes of this lecture

Through this lecture, students will aim to be able to:

- Handle various neural networks (NNs) under the general definition of the NN as a **directed acyclic graph (DAG)**.
- Describe the specific function represented by a general neural network from its DAG structure, activation function, edge-parameter correspondence.

Learning outcomes of this lecture

Through this lecture, students will aim to be able to:

- Handle various neural networks (NNs) under the general definition of the NN as a **directed acyclic graph (DAG)**.
- Describe the specific function represented by a general neural network from its DAG structure, activation function, edge-parameter correspondence.
- State the definitions of frequently used terms in modern neural network applications, such as **architecture**, **checkpoint**, and **model**.

Policy of this lecture

Although many students may have already studied neural networks, this lecture will deliberately start from their mathematical definition. The goal is to **define neural networks in the most general form possible**.

Policy of this lecture

Although many students may have already studied neural networks, this lecture will deliberately start from their mathematical definition. The goal is to **define neural networks in the most general form possible**.

The field of AI and machine learning is developing very rapidly, and the specific models widely known today may well be outdated by the time you graduate.

Policy of this lecture

Although many students may have already studied neural networks, this lecture will deliberately start from their mathematical definition. The goal is to **define neural networks in the most general form possible**.

The field of AI and machine learning is developing very rapidly, and the specific models widely known today may well be outdated by the time you graduate.

Therefore, this lecture emphasizes understanding more general and universal concepts that will be applicable in the future and will not become obsolete.

Policy of this lecture

One might ask, "Is a fundamental understanding of neural networks necessary if we are to use them as black boxes?" The answer to this question is "Yes."

Policy of this lecture

One might ask, "Is a fundamental understanding of neural networks necessary if we are to use them as black boxes?" The answer to this question is "Yes."

In particular, the **distinction between architecture and checkpoint** that we will learn in this lecture is extremely important for using AI in the real world.

Policy of this lecture

One might ask, "Is a fundamental understanding of neural networks necessary if we are to use them as black boxes?" The answer to this question is "Yes."

In particular, the **distinction between architecture and checkpoint** that we will learn in this lecture is extremely important for using AI in the real world.

Without understanding this distinction, you will not be able to correctly understand the issue of AI **licenses**, which we will cover in the next lecture. Using AI without regard for its license can lead to legal risks such as copyright infringement.

Outline

1. Introduction

2. Preparation: Mathematical Notations

3. What does “neural network” truly refer to? - DAG-based parametric function

4. Does the DAG-based definition include practical neural networks?

5. Architecture and Checkpoints

6. Summary and Future Outlook

Outline

1. Introduction

2. Preparation: Mathematical Notations

3. What does “neural network” truly refer to? - DAG-based parametric function

4. Does the DAG-based definition include practical neural networks?

5. Architecture and Checkpoints

6. Summary and Future Outlook

Preparation: mathematical notations

Here is a review of the basic mathematical notations used in this lecture.

- **Definition:**

- $(\text{LHS}) := (\text{RHS})$: The left-hand side is defined by the right-hand side.

Preparation: mathematical notations

Here is a review of the basic mathematical notations used in this lecture.

- **Definition:**

- $(\text{LHS}) := (\text{RHS})$: The left-hand side is defined by the right-hand side.

- **Set:** Denoted by uppercase calligraphic letters (e.g., $\mathcal{A}$).

- $x \in \mathcal{A}$: element x belongs to set $\mathcal{A}$.
- $\{\}$: The empty set.
- $\{a, b, c\}$: The set consisting of elements a, b, c .
- $\{x \in \mathcal{A} | P(x)\}$: The set of elements in $\mathcal{A}$ for which $P(x)$ is true.
- $\mathbb{R}$ (reals), $\mathbb{Z}$ (integers), with subscripts like $>_0$ (positive) or $\geq_0$ (non-negative).
- $[1, k]_{\mathbb{Z}} := \{1, 2, \dots, k\}$.

Preparation: mathematical notations

- **Function:**

- $f : X \rightarrow Y$: f is a map from set X to set Y .
- $y = f(x)$: The output of f for input $x \in X$ is $y \in Y$.

Preparation: mathematical notations

- **Function:**

- $f : X \rightarrow Y$: f is a map from set X to set Y .
- $y = f(x)$: The output of f for input $x \in X$ is $y \in Y$.

- **Vector:** A column of numbers, denoted by bold italic lowercase letters (e.g., $\mathbf{v} \in \mathbb{R}^n$). The i -th element is v_i .

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}, \quad \mathbf{v}^\top = \begin{bmatrix} v_1 & v_2 & \cdots & v_n \end{bmatrix}$$

Preparation: mathematical notations

- **Function:**

- $f : X \rightarrow Y$: f is a map from set X to set Y .
- $y = f(x)$: The output of f for input $x \in X$ is $y \in Y$.

- **Vector:** A column of numbers, denoted by bold italic lowercase letters (e.g., $\mathbf{v} \in \mathbb{R}^n$). The i -th element is v_i .

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}, \quad \mathbf{v}^\top = \begin{bmatrix} v_1 & v_2 & \cdots & v_n \end{bmatrix}$$

- **Matrix:** Denoted by bold italic uppercase letters (e.g., $\mathbf{A} \in \mathbb{R}^{m,n}$). The element in the i -th row, j -th column is $a_{i,j}$.

Preparation: mathematical notations

- **Function:**

- $f : X \rightarrow Y$: f is a map from set X to set Y .
- $y = f(x)$: The output of f for input $x \in X$ is $y \in Y$.

- **Vector:** A column of numbers, denoted by bold italic lowercase letters (e.g., $\mathbf{v} \in \mathbb{R}^n$). The i -th element is v_i .

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}, \quad \mathbf{v}^\top = \begin{bmatrix} v_1 & v_2 & \cdots & v_n \end{bmatrix}$$

- **Matrix:** Denoted by bold italic uppercase letters (e.g., $\mathbf{A} \in \mathbb{R}^{m,n}$). The element in the i -th row, j -th column is $a_{i,j}$.
- **Tensor:** In this lecture, simply a multi-dimensional array.
 - Vector = 1st-order tensor, Matrix = 2nd-order tensor.
 - Higher-order tensors are denoted by $\underline{\mathbf{A}}$.

Outline

1. Introduction

2. Preparation: Mathematical Notations

3. What does “neural network” truly refer to? - DAG-based parametric function

4. Does the DAG-based definition include practical neural networks?

5. Architecture and Checkpoints

6. Summary and Future Outlook

Outline

1. Introduction

2. Preparation: Mathematical Notations

3. What does “neural network” truly refer to? - DAG-based parametric function

4. Does the DAG-based definition include practical neural networks?

5. Architecture and Checkpoints

6. Summary and Future Outlook

Question: Totally different kinds of parametric functions are called “neural networks” in practice (e.g., Transformer, U-net, LSTM) depending on applications. What

Question: Totally different kinds of parametric functions are called “neural networks” in practice (e.g., Transformer, U-net, LSTM) depending on applications. What does **the term “neural network” truly refer to?** When

Question: Totally different kinds of parametric functions are called “neural networks” in practice (e.g., Transformer, U-net, LSTM) depending on applications. What does **the term “neural network” truly refer to**? When can we apply the neural network theory to a new parametric function?

MLP is a neural network, but we don't use it.

The most classical neural network is the multilayer perceptron (MLP):

MLP is a neural network, but we don't use it.

The most classical neural network is the multilayer perceptron (MLP):

$$f_{\theta} = g^{[L]}(\mathbf{b}^{[L]} + \mathbf{W}^{[L]}g^{[L-1]}(\mathbf{b}^{[L-1]} + \mathbf{W}^{[L-1]}g^{[L-2]}(\dots g^{[1]}(\mathbf{b}^{[1]} + \mathbf{W}^{[1]}\mathbf{x})))),$$

where the parameter is $\theta = (\mathbf{W}^{[1]}, \mathbf{b}^{[1]}, \mathbf{W}^{[2]}, \mathbf{b}^{[2]}, \dots, \mathbf{W}^{[L]}, \mathbf{b}^{[L]})$.

However,

MLP is a neural network, but we don't use it.

The most classical neural network is the multilayer perceptron (MLP):

$$f_{\theta} = g^{[L]}(\mathbf{b}^{[L]} + \mathbf{W}^{[L]}g^{[L-1]}(\mathbf{b}^{[L-1]} + \mathbf{W}^{[L-1]}g^{[L-2]}(\dots g^{[1]}(\mathbf{b}^{[1]} + \mathbf{W}^{[1]}\mathbf{x})))),$$

where the parameter is $\theta = (\mathbf{W}^{[1]}, \mathbf{b}^{[1]}, \mathbf{W}^{[2]}, \mathbf{b}^{[2]}, \dots, \mathbf{W}^{[L]}, \mathbf{b}^{[L]})$.

However, we do not use MLP in generative AIs. Instead, we use

MLP is a neural network, but we don't use it.

The most classical neural network is the multilayer perceptron (MLP):

$$f_{\theta} = g^{[L]}(\mathbf{b}^{[L]} + \mathbf{W}^{[L]}g^{[L-1]}(\mathbf{b}^{[L-1]} + \mathbf{W}^{[L-1]}g^{[L-2]}(\dots g^{[1]}(\mathbf{b}^{[1]} + \mathbf{W}^{[1]}\mathbf{x})))),$$

where the parameter is $\theta = (\mathbf{W}^{[1]}, \mathbf{b}^{[1]}, \mathbf{W}^{[2]}, \mathbf{b}^{[2]}, \dots, \mathbf{W}^{[L]}, \mathbf{b}^{[L]})$.

However, we do not use MLP in generative AIs. Instead, we use

- Transformer in natural language generation.

MLP is a neural network, but we don't use it.

The most classical neural network is the multilayer perceptron (MLP):

$$f_{\theta} = g^{[L]}(\mathbf{b}^{[L]} + \mathbf{W}^{[L]}g^{[L-1]}(\mathbf{b}^{[L-1]} + \mathbf{W}^{[L-1]}g^{[L-2]}(\dots g^{[1]}(\mathbf{b}^{[1]} + \mathbf{W}^{[1]}\mathbf{x})))),$$

where the parameter is $\theta = (\mathbf{W}^{[1]}, \mathbf{b}^{[1]}, \mathbf{W}^{[2]}, \mathbf{b}^{[2]}, \dots, \mathbf{W}^{[L]}, \mathbf{b}^{[L]})$.

However, we do not use MLP in generative AIs. Instead, we use

- Transformer in natural language generation.
- Convolutional neural networks in image generation.

MLP is a neural network, but we don't use it.

The most classical neural network is the multilayer perceptron (MLP):

$$f_{\theta} = g^{[L]}(\mathbf{b}^{[L]} + \mathbf{W}^{[L]}g^{[L-1]}(\mathbf{b}^{[L-1]} + \mathbf{W}^{[L-1]}g^{[L-2]}(\dots g^{[1]}(\mathbf{b}^{[1]} + \mathbf{W}^{[1]}\mathbf{x})))),$$

where the parameter is $\theta = (\mathbf{W}^{[1]}, \mathbf{b}^{[1]}, \mathbf{W}^{[2]}, \mathbf{b}^{[2]}, \dots, \mathbf{W}^{[L]}, \mathbf{b}^{[L]})$.

However, we do not use MLP in generative AIs. Instead, we use

- Transformer in natural language generation.
- Convolutional neural networks in image generation.

Long short term memories, gated recurrent neural networks, etc. are also used.

The above models are all called “neural networks,” while

MLP is a neural network, but we don't use it.

The most classical neural network is the multilayer perceptron (MLP):

$$f_{\theta} = g^{[L]}(\mathbf{b}^{[L]} + \mathbf{W}^{[L]}g^{[L-1]}(\mathbf{b}^{[L-1]} + \mathbf{W}^{[L-1]}g^{[L-2]}(\dots g^{[1]}(\mathbf{b}^{[1]} + \mathbf{W}^{[1]}\mathbf{x})))),$$

where the parameter is $\theta = (\mathbf{W}^{[1]}, \mathbf{b}^{[1]}, \mathbf{W}^{[2]}, \mathbf{b}^{[2]}, \dots, \mathbf{W}^{[L]}, \mathbf{b}^{[L]})$.

However, we do not use MLP in generative AIs. Instead, we use

- Transformer in natural language generation.
- Convolutional neural networks in image generation.

Long short term memories, gated recurrent neural networks, etc. are also used.

The above models are all called “neural networks,” while other models, e.g., k-NN, random forests, gradient boosted decision trees, etc. are not called so.

MLP is a neural network, but we don't use it.

The most classical neural network is the multilayer perceptron (MLP):

$$f_{\theta} = g^{[L]}(\mathbf{b}^{[L]} + \mathbf{W}^{[L]}g^{[L-1]}(\mathbf{b}^{[L-1]} + \mathbf{W}^{[L-1]}g^{[L-2]}(\dots g^{[1]}(\mathbf{b}^{[1]} + \mathbf{W}^{[1]}\mathbf{x}))),$$

where the parameter is $\theta = (\mathbf{W}^{[1]}, \mathbf{b}^{[1]}, \mathbf{W}^{[2]}, \mathbf{b}^{[2]}, \dots, \mathbf{W}^{[L]}, \mathbf{b}^{[L]})$.

However, we do not use MLP in generative AIs. Instead, we use

- Transformer in natural language generation.
- Convolutional neural networks in image generation.

Long short term memories, gated recurrent neural networks, etc. are also used.

The above models are all called “neural networks,” while other models, e.g., k-NN, random forests, gradient boosted decision trees, etc. are not called so.

What does the term “neural network” refers to?

This is not just a convention problem but affects real applications.

Practical questions in reality: Assume we face a new task and want to use a new parametric function.

This is not just a convention problem but affects real applications.

Practical questions in reality: Assume we face a new task and want to use a new parametric function.

- Does it have a good property as existing “neural networks” have?

This is not just a convention problem but affects real applications.

Practical questions in reality: Assume we face a new task and want to use a new parametric function.

- Does it have a good property as existing “neural networks” have?
- Can it allow us to apply existing neural network theory?

This is not just a convention problem but affects real applications.

Practical questions in reality: Assume we face a new task and want to use a new parametric function.

- Does it have a good property as existing “neural networks” have?
- Can it allow us to apply existing neural network theory?

To answer the above questions in the future, we need a formal and general definition of the neural network.

General definition of neural network

An **Artificial neural network (ANN)** (feedforward neural networks) is a parametric function defined by

General definition of neural network

An **Artificial neural network (ANN)** (feedforward neural networks) is a parametric function defined by a **directed acyclic graph (DAG)**, where

General definition of neural network

An **Artificial neural network (ANN)** (feedforward neural networks) is a parametric function defined by a **directed acyclic graph (DAG)**, where

- each node represents a fixed function.

General definition of neural network

An **Artificial neural network (ANN)** (feedforward neural networks) is a parametric function defined by a **directed acyclic graph (DAG)**, where

- each node represents a fixed function.
- each edge is associated with a real number parameter and represents the multiplication operation.

General definition of neural network

An **Artificial neural network (ANN)** (feedforward neural networks) is a parametric function defined by a **directed acyclic graph (DAG)**, where

- each node represents a fixed function.
- each edge is associated with a real number parameter and represents the multiplication operation.

The parametric function, as a whole, is defined as the **composite function** of the functions represented by nodes and edges.

What motivates the construction of the NN?

You can understand the construction of the NN in the following perspectives.

What motivates the construction of the NN?

You can understand the construction of the NN in the following perspectives.

- Since any (directed acyclic) graph structure is allowed, we can choose the best one depending on our task.

What motivates the construction of the NN?

You can understand the construction of the NN in the following perspectives.

- Since any (directed acyclic) graph structure is allowed, we can choose the best one depending on our task.
- Since each node is a non-linear function, as a whole, the neural network can represent complicated non-linear the input-output.

What motivates the construction of the NN?

You can understand the construction of the NN in the following perspectives.

- Since any (directed acyclic) graph structure is allowed, we can choose the best one depending on our task.
- Since each node is a non-linear function, as a whole, the neural network can represent complicated non-linear the input-output.
- Since each edge is associated with a parameter, various functions are realized by changing parameters.

What motivates the construction of the NN?

You can understand the construction of the NN in the following perspectives.

- Since any (directed acyclic) graph structure is allowed, we can choose the best one depending on our task.
- Since each node is a non-linear function, as a whole, the neural network can represent complicated non-linear the input-output.
- Since each edge is associated with a parameter, various functions are realized by changing parameters.

Let's look at each component.

Remarks on the terminology

Remark (On Terminology)

"Neural network" originally refers to biological neural circuitry. Engineering models are "artificial neural networks." However, in modern AI, the two are best understood as independent concepts. In this lecture, "neural network" will mean an artificial one.

Remarks on the terminology

Remark (On Terminology)

"Neural network" originally refers to biological neural circuitry. Engineering models are "artificial neural networks." However, in modern AI, the two are best understood as independent concepts. In this lecture, "neural network" will mean an artificial one.

Remark (Scope of this Lecture)

All networks in this lecture are **feedforward neural networks** (e.g., MLP, CNN, Transformer). We will not cover non-feedforward networks like Boltzmann machines.

Starting with a directed graph

Defining a (parametric) function means defining the output for all possible input.

Starting with a directed graph

Defining a (parametric) function means defining the output for all possible input.

We want to represent a complicated input-output relation by iteratively combining elementary functions.

Starting with a directed graph

Defining a (parametric) function means defining the output for all possible input.

We want to represent a complicated input-output relation by iteratively combining elementary functions.

Hence, it is natural to use a **directed** graph, whose direction is from the input towards the output.

Starting with a directed graph

Defining a (parametric) function means defining the output for all possible input.

We want to represent a complicated input-output relation by iteratively combining elementary functions.

Hence, it is natural to use a **directed** graph, whose direction is from the input towards the output.

Definition (Directed Multigraph)

A **directed multigraph** G is a tuple $G = (\mathcal{V}, \mathcal{E}, \text{Tail}, \text{Head})$, consisting of:

- a set of **nodes** or **vertices** $\mathcal{V}$,
- a set of **edges** or **directed arcs** $\mathcal{E}$,
- and two maps, $\text{Tail} : \mathcal{E} \rightarrow \mathcal{V}$ and $\text{Head} : \mathcal{E} \rightarrow \mathcal{V}$, which assign a tail and head node to each edge.

Directed graph terminologies

We can define relationships between nodes:

- **Incoming edges** of a node v : $\mathcal{E}_{\text{in}}^{(v)} = \{e \in \mathcal{E} \mid \text{Head}(e) = v\}$.
- **Outgoing edges** of a node v : $\mathcal{E}_{\text{out}}^{(v)} = \{e \in \mathcal{E} \mid \text{Tail}(e) = v\}$.
- A **parent node** of v is the tail of an incoming edge.
- A **child node** of v is the head of an outgoing edge.

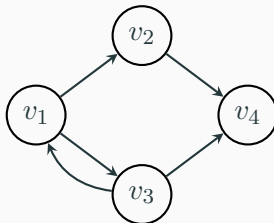


Figure 1: Example of a directed graph. Here, v_1 is a parent of v_2 , and v_4 is a child of v_3 .

Directed acyclic graph, avoiding a circular reference

To define the output without a circular reference, we want to avoid a **cycle**.

Directed acyclic graph, avoiding a circular reference

To define the output without a circular reference, we want to avoid a **cycle**.

Definition (Path and Cycle)

In a directed multigraph G :

- A **path** is a sequence of nodes and edges, where each edge connects a node to the next one in the sequence.
- A **cycle** is a path that starts and ends at the same node.

Directed acyclic graph, avoiding a circular reference

To define the output without a circular reference, we want to avoid a **cycle**.

Definition (Path and Cycle)

In a directed multigraph G :

- A **path** is a sequence of nodes and edges, where each edge connects a node to the next one in the sequence.
- A **cycle** is a path that starts and ends at the same node.

Definition (Directed Acyclic Graph)

When a directed multigraph G has **no cycles**, it is called a **Directed Acyclic Graph (DAG)**.

Directed acyclic graph, avoiding a circular reference

To define the output without a circular reference, we want to avoid a **cycle**.

Definition (Path and Cycle)

In a directed multigraph G :

- A **path** is a sequence of nodes and edges, where each edge connects a node to the next one in the sequence.
- A **cycle** is a path that starts and ends at the same node.

Definition (Directed Acyclic Graph)

When a directed multigraph G has **no cycles**, it is called a **Directed Acyclic Graph (DAG)**.

Intuitively, a DAG allows only "one-way" flows. This means the nodes can be **topologically sorted**.

Directed acyclic graph (DAG)

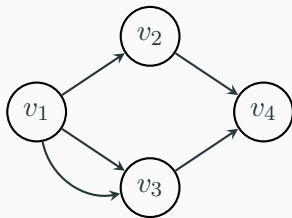


Figure 2: Example of a DAG (including multi-edges)

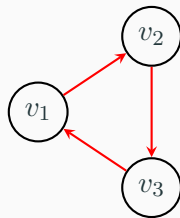


Figure 3: Example that is not a DAG (a cycle $v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_1$ exists)

Directed acyclic graph (DAG)

In a DAG, there is always at least one node with no incoming edges.

Directed acyclic graph (DAG)

In a DAG, there is always at least one node with no incoming edges.

These nodes become the **input nodes** that receive the input when the neural network is viewed as a function.

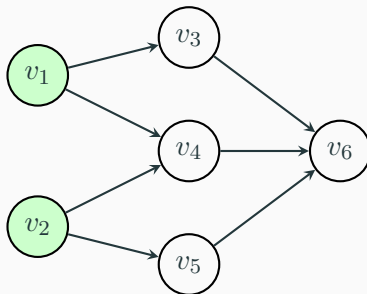


Figure 4: Example of a DAG. Nodes v_1, v_2 (green) are the input nodes.

The function represented by a neural network

A neural network is a collection of simple computational units (nodes and edges) connected in a DAG structure. The entire network represents a massive composite function.

The function represented by a neural network

A neural network is a collection of simple computational units (nodes and edges) connected in a DAG structure. The entire network represents a massive composite function.

- **Nodes:** Receive values from parent nodes or network inputs, apply a fixed **activation function**, and compute a new value.

The function represented by a neural network

A neural network is a collection of simple computational units (nodes and edges) connected in a DAG structure. The entire network represents a massive composite function.

- **Nodes:** Receive values from parent nodes or network inputs, apply a fixed **activation function**, and compute a new value.
- **Edges:** Receive a value from their tail node, multiply it by a parameter value (a **weight**), and pass it to their head node.

The function represented by a neural network

First, we define the parameter-independent part, the **architecture**.

Definition (Neural Network Architecture)

A neural network architecture is rigorously defined by a tuple including:

1. **Computation Graph:** A DAG $G = (\mathcal{V}, \mathcal{E}, \text{Tail}, \text{Head})$, where nodes are topologically sorted integers.
2. **Input/Output Dimensions:** The overall network's input dimension d_{input} and output dimension d_{output} .
3. **Node Specifications:** For each node $v \in \mathcal{V}$:
 - An **activation function** $g^{(v)} : \mathbb{R}^{d_{\text{arg}}^{(v)}} \rightarrow \mathbb{R}^{d_{\text{ret}}^{(v)}}$.
 - An **argument source tuple** $\text{NodeSrc}^{(v)}$, specifying where each argument for $g^{(v)}$ comes from (either a network input or an incoming edge).

The function represented by a neural network

Definition (Neural Network Architecture (cont.))

4. **Edge Specifications:** For each edge $e \in \mathcal{E}$, an **edge source index** $\text{EdgeSrc}(e)$ indicating which component of its tail node's return vector to use.
5. **Parameter Specifications:**
 - The total number of parameters d_{param} .
 - A **weight map** $\text{Weight} : \mathcal{E} \rightarrow [1, d_{\text{param}}]_{\mathbb{Z}}$ that assigns a parameter to each edge (allowing for **parameter sharing**).
 - Sets of trainable (**variable**) and non-trainable (**fixed**) parameters.
6. **Output Specifications:** An **output source map** OutSrc specifying which component of which node's return vector corresponds to each component of the overall network output.

The function represented by a neural network

Definition (The Function Represented by a Neural Network)

Given an architecture and specific parameter values (a **checkpoint**) $\theta \in \mathbb{R}^{d_{\text{param}}}$, the function $f_{\theta} : \mathbb{R}^{d_{\text{input}}} \rightarrow \mathbb{R}^{d_{\text{output}}}$ is computed as follows for an input x :

1. **Per-Node Computation:** In topological order ($v \in \mathcal{V}$):
 - 1.1 Construct the argument vector $\mathbf{a}^{(v)}$ for node v 's activation function $g^{(v)}$. Each argument is taken from either a network input x_j or a weighted value from a parent node, $\theta_p \times r_k^{(u)}$.
 - 1.2 Compute the return value vector for node v : $\mathbf{r}^{(v)} := g^{(v)}(\mathbf{a}^{(v)})$.
2. **Overall Network Output:** After all nodes are computed, construct the output vector $\mathbf{y}$ by picking the specified return values from the nodes.

The function represented by a neural network

Remark (On Generality)

This definition is very general.

- It can represent common operations like the weighted sum in MLPs by defining the activation function appropriately (e.g., $g^{(v)}(\mathbf{a}) = \text{ReLU}(\sum_i a_i)$).
- It can also represent complex layers like convolutions in CNNs and attention in Transformers.
- With mild constraints on the activation functions (e.g., local Lipschitz continuity), the parameters for this entire general class of functions can be trained using a unified method (backpropagation + gradient descent).

This means we don't need a separate training theory for each new architecture.

Neural network definition - answer

Question: What

Neural network definition - answer

Question: What does the term “neural network” truly refer to? When can we apply the neural network theory to a new parametric function?

Answer:

Neural network definition - answer

Question: What does the term “neural network” truly refer to? When can we apply the neural network theory to a new parametric function?

Answer: A neural network is a parametric function defined by a directed acyclic graph.

Outline

1. Introduction

2. Preparation: Mathematical Notations

3. What does “neural network” truly refer to? - DAG-based parametric function

4. Does the DAG-based definition include practical neural networks?

5. Architecture and Checkpoints

6. Summary and Future Outlook

Outline

1. Introduction
2. Preparation: Mathematical Notations
3. What does “neural network” truly refer to? - DAG-based parametric function
- 4. Does the DAG-based definition include practical neural networks?**
5. Architecture and Checkpoints
6. Summary and Future Outlook

Question: Does the definition really include practical neural networks?

Specific computation examples

Let's look at a few examples to see how this formal definition can represent various functions.

Specific computation examples

Let's look at a few examples to see how this formal definition can represent various functions.

The key takeaways from this section are:

- Simple models like linear and logistic regression are just special cases of neural networks. A neural network is a broad concept that includes these.

Specific computation examples

Let's look at a few examples to see how this formal definition can represent various functions.

The key takeaways from this section are:

- Simple models like linear and logistic regression are just special cases of neural networks. A neural network is a broad concept that includes these.
- The architecture of a neural network can be very flexible and irregular. The design freedom is immense, opening the door for future innovations.

Specific computation examples

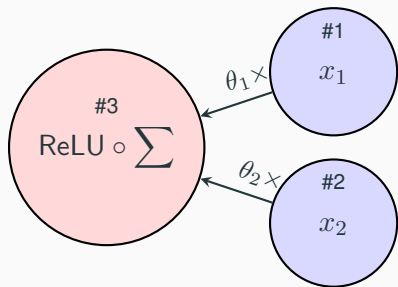
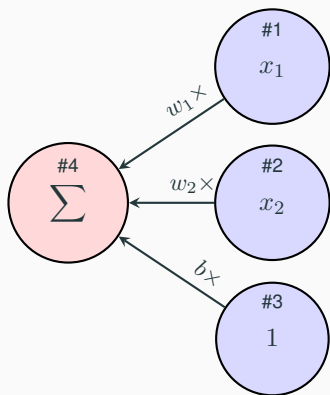


Figure 5: Legend for DAG representation of a neural network.

- **Nodes:** Circles are computational units. **Blue** for input, **red** for output. Inside is the node number and activation function.
- **Activation Function Shorthand:** A function name like 'ReLU' on a node with multiple inputs is shorthand for "sum the inputs, then apply the function" (e.g., $\text{ReLU} \circ \Sigma$).
- **Edges:** Arrows show data flow. A label like $\theta_1 \times$ means multiply by parameter θ_1 .

Example: linear regression

Consider the linear regression model $f(\mathbf{x}) = w_1x_1 + w_2x_2 + b$. This can be represented by the following architecture.



Architecture:

- **Nodes:** 1, 2, 3 for inputs (x_1, x_2 , and a constant 1 for the bias).
- **Node 4:** Output node. Activation function is summation (Σ).
- **Edges:** Weights are parameters w_1, w_2, b .

Figure 6: DAG representation of a linear regression

Example: linear regression

Let's execute the computation.

- Checkpoint: $\theta = [w_1, w_2, b]^\top = [1.0, -2.0, 0.5]^\top$.
- Input: $x = [x_1, x_2]^\top = [3.0, 4.0]^\top$.

Example: linear regression

Let's execute the computation.

- Checkpoint: $\theta = [w_1, w_2, b]^\top = [1.0, -2.0, 0.5]^\top$.
- Input: $x = [x_1, x_2]^\top = [3.0, 4.0]^\top$.

1. Nodes 1, 2, 3: These are input nodes.

- Return value of node 1: $r^{(1)} = [x_1] = [3.0]$.
- Return value of node 2: $r^{(2)} = [x_2] = [4.0]$.
- Return value of node 3: $r^{(3)} = [1.0]$ (constant bias input).

Example: linear regression

2. **Node 4:** This node sums its arguments. First, we compute the arguments.

- Arg 1 (from node 1): $a_1^{(4)} = w_1 \times r_1^{(1)} = 1.0 \times 3.0 = 3.0$.
- Arg 2 (from node 2): $a_2^{(4)} = w_2 \times r_1^{(2)} = -2.0 \times 4.0 = -8.0$.
- Arg 3 (from node 3): $a_3^{(4)} = b \times r_1^{(3)} = 0.5 \times 1.0 = 0.5$.

Example: linear regression

2. **Node 4:** This node sums its arguments. First, we compute the arguments.

- Arg 1 (from node 1): $a_1^{(4)} = w_1 \times r_1^{(1)} = 1.0 \times 3.0 = 3.0$.
- Arg 2 (from node 2): $a_2^{(4)} = w_2 \times r_1^{(2)} = -2.0 \times 4.0 = -8.0$.
- Arg 3 (from node 3): $a_3^{(4)} = b \times r_1^{(3)} = 0.5 \times 1.0 = 0.5$.

The return value of node 4 is the sum of these arguments:

$$\mathbf{r}^{(4)} = g^{(4)}(3.0, -8.0, 0.5) = [3.0 - 8.0 + 0.5] = [-4.5]$$

Example: linear regression

2. **Node 4:** This node sums its arguments. First, we compute the arguments.

- Arg 1 (from node 1): $a_1^{(4)} = w_1 \times r_1^{(1)} = 1.0 \times 3.0 = 3.0$.
- Arg 2 (from node 2): $a_2^{(4)} = w_2 \times r_1^{(2)} = -2.0 \times 4.0 = -8.0$.
- Arg 3 (from node 3): $a_3^{(4)} = b \times r_1^{(3)} = 0.5 \times 1.0 = 0.5$.

The return value of node 4 is the sum of these arguments:

$$\mathbf{r}^{(4)} = g^{(4)}(3.0, -8.0, 0.5) = [3.0 - 8.0 + 0.5] = [-4.5]$$

Final Output: The network's output is taken from node 4, so $\mathbf{y} = [-4.5]$.

Example: linear regression

Exercise

For parameters $w = \begin{bmatrix} -0.5 \\ 3.0 \end{bmatrix}$, $b = -1.0$, find the output for the same input

$$x = \begin{bmatrix} 3.0 \\ 4.0 \end{bmatrix}.$$

Example: linear regression

Exercise

For parameters $w = \begin{bmatrix} -0.5 \\ 3.0 \end{bmatrix}$, $b = -1.0$, find the output for the same input

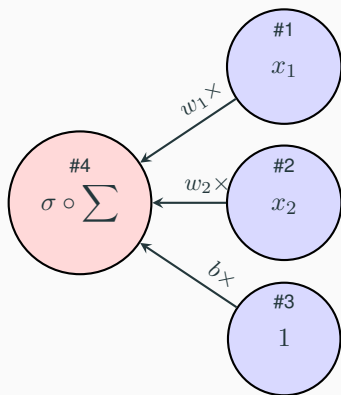
$$x = \begin{bmatrix} 3.0 \\ 4.0 \end{bmatrix}.$$

Solution:

- The arguments to node 4 are:
 - $a_1^{(4)} = -0.5 \times 3.0 = -1.5$.
 - $a_2^{(4)} = 3.0 \times 4.0 = 12.0$.
 - $a_3^{(4)} = -1.0 \times 1.0 = -1.0$.
- The sum is $r^{(4)} = [-1.5 + 12.0 - 1.0] = [9.5]$.
- Final Output: $y = [9.5]$.

Example: logistic regression

Consider the logistic regression model $f(\mathbf{x}) = \sigma(w_1x_1 + w_2x_2 + b)$, where σ is the sigmoid function.



Architecture: Almost the same as linear regression. The only change is the activation function of node 4.

- **Node 4 Activation:**

$$g^{(4)}(\mathbf{a}) = \sigma(\sum_i a_i).$$

Figure 7: DAG representation of a logistic regression

Example: logistic regression

Let's execute the computation.

- Checkpoint: $\theta = [1.0, -2.0, 0.5]^\top$.
- Input: $x = [3.0, 4.0]^\top$.

Example: logistic regression

Let's execute the computation.

- Checkpoint: $\theta = [1.0, -2.0, 0.5]^\top$.
- Input: $x = [3.0, 4.0]^\top$.

The computation of the arguments for node 4 is identical to the linear regression case.

$$a^{(4)} = [3.0, -8.0, 0.5]^\top$$

The sum is $3.0 - 8.0 + 0.5 = -4.5$.

Example: logistic regression

Let's execute the computation.

- Checkpoint: $\theta = [1.0, -2.0, 0.5]^\top$.
- Input: $x = [3.0, 4.0]^\top$.

The computation of the arguments for node 4 is identical to the linear regression case.

$$\mathbf{a}^{(4)} = [3.0, -8.0, 0.5]^\top$$

The sum is $3.0 - 8.0 + 0.5 = -4.5$.

The return value of node 4 applies the sigmoid function:

$$\begin{aligned} \mathbf{r}^{(4)} &= g^{(4)}(\mathbf{a}^{(4)}) = [\sigma(-4.5)] \\ \sigma(-4.5) &= \frac{1}{1 + e^{4.5}} \approx \frac{1}{1 + 90.017} \approx 0.01098 \end{aligned}$$

Example: logistic regression

Let's execute the computation.

- Checkpoint: $\theta = [1.0, -2.0, 0.5]^\top$.
- Input: $x = [3.0, 4.0]^\top$.

The computation of the arguments for node 4 is identical to the linear regression case.

$$\mathbf{a}^{(4)} = [3.0, -8.0, 0.5]^\top$$

The sum is $3.0 - 8.0 + 0.5 = -4.5$.

The return value of node 4 applies the sigmoid function:

$$\begin{aligned} \mathbf{r}^{(4)} &= g^{(4)}(\mathbf{a}^{(4)}) = [\sigma(-4.5)] \\ \sigma(-4.5) &= \frac{1}{1 + e^{4.5}} \approx \frac{1}{1 + 90.017} \approx 0.01098 \end{aligned}$$

Final Output: $y \approx [0.01098]$.

Example: logistic regression

Exercise

For parameters $w = \begin{bmatrix} -0.5 \\ 3.0 \end{bmatrix}$, $b = -1.0$, find the output of the logistic regression model for the input $x = \begin{bmatrix} 3.0 \\ 4.0 \end{bmatrix}$.

Example: logistic regression

Exercise

For parameters $w = \begin{bmatrix} -0.5 \\ 3.0 \end{bmatrix}$, $b = -1.0$, find the output of the logistic regression model for the input $x = \begin{bmatrix} 3.0 \\ 4.0 \end{bmatrix}$.

Solution:

- The sum of arguments for node 4 is 9.5 (from the linear regression exercise).
- The return value is $r^{(4)} = [\sigma(9.5)]$.
- $\sigma(9.5) = \frac{1}{1+e^{-9.5}} \approx \frac{1}{1+0.0000748} \approx 0.999925$.
- Final Output: $y \approx [0.999925]$.

Example: multilayer perceptron (MLP)

Consider an MLP with a 2D input, a 2D hidden layer (with ReLU), and a 1D output.

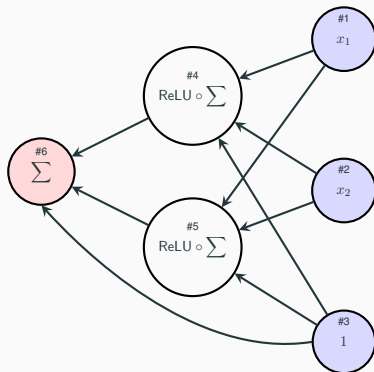


Figure 8: DAG of an MLP. Note the intermediate nodes #4 and #5.

Example: multilayer perceptron (MLP)

Let's execute the computation.

- Checkpoint:

$$\theta = [w_{11}, w_{12}, w_{21}, w_{22}, b_1, b_2, v_1, v_2, c]^T = [1, 1, -1, -1, 0, 1, 2, 3, -1]^T.$$

- Input: $x = [3.0, 4.0]^T$.

Example: multilayer perceptron (MLP)

Let's execute the computation.

- Checkpoint:

$$\theta = [w_{11}, w_{12}, w_{21}, w_{22}, b_1, b_2, v_1, v_2, c]^T = [1, 1, -1, -1, 0, 1, 2, 3, -1]^T.$$

- Input: $x = [3.0, 4.0]^T$.

1. **Nodes 1, 2, 3:** Return input values $[3.0]$, $[4.0]$, $[1.0]$ respectively.

Example: multilayer perceptron (MLP)

Let's execute the computation.

- Checkpoint:

$$\theta = [w_{11}, w_{12}, w_{21}, w_{22}, b_1, b_2, v_1, v_2, c]^T = [1, 1, -1, -1, 0, 1, 2, 3, -1]^T.$$

- Input: $x = [3.0, 4.0]^T$.

1. **Nodes 1, 2, 3:** Return input values $[3.0]$, $[4.0]$, $[1.0]$ respectively.

2. **Node 4 (Hidden 1):**

- Sum of arguments: $(1 \times 3.0) + (1 \times 4.0) + (0 \times 1.0) = 7.0$.
- Return value: $r^{(4)} = [\text{ReLU}(7.0)] = [7.0]$.

Example: multilayer perceptron (MLP)

3. Node 5 (Hidden 2):

- Sum of arguments: $(-1 \times 3.0) + (-1 \times 4.0) + (1 \times 1.0) = -3.0 - 4.0 + 1.0 = -6.0$.
- Return value: $r^{(5)} = [\text{ReLU}(-6.0)] = [0.0]$.

Example: multilayer perceptron (MLP)

3. Node 5 (Hidden 2):

- Sum of arguments: $(-1 \times 3.0) + (-1 \times 4.0) + (1 \times 1.0) = -3.0 - 4.0 + 1.0 = -6.0$.
- Return value: $r^{(5)} = [\text{ReLU}(-6.0)] = [0.0]$.

4. Node 6 (Output):

- Arguments come from nodes 4, 5, and the bias node 3.
- Sum of arguments:
 $(v_1 \times r_1^{(4)}) + (v_2 \times r_1^{(5)}) + (c \times r_1^{(3)}) = (2 \times 7.0) + (3 \times 0.0) + (-1 \times 1.0)$.
- $= 14.0 + 0.0 - 1.0 = 13.0$.
- Return value: $r^{(6)} = [13.0]$.

Example: multilayer perceptron (MLP)

3. Node 5 (Hidden 2):

- Sum of arguments: $(-1 \times 3.0) + (-1 \times 4.0) + (1 \times 1.0) = -3.0 - 4.0 + 1.0 = -6.0$.
- Return value: $r^{(5)} = [\text{ReLU}(-6.0)] = [0.0]$.

4. Node 6 (Output):

- Arguments come from nodes 4, 5, and the bias node 3.
- Sum of arguments:
 $(v_1 \times r_1^{(4)}) + (v_2 \times r_1^{(5)}) + (c \times r_1^{(3)}) = (2 \times 7.0) + (3 \times 0.0) + (-1 \times 1.0)$.
- $= 14.0 + 0.0 - 1.0 = 13.0$.
- Return value: $r^{(6)} = [13.0]$.

Final Output: $y = [13.0]$.

Example: multilayer perceptron (MLP)

Remark (Vector Notation for MLP)

The MLP's computation can be expressed concisely using vectors and matrices. Let the input be x , hidden weights be W , hidden bias be b , output weights be v , and output bias be c .

$$y = c + v^{\top} \text{ReLU}(b + Wx) \quad (1)$$

Here, ReLU applied to a vector means it's applied to each element (**element-wise application**).

Example: multilayer perceptron (MLP)

Exercise

In the MLP example, find the output for the input $x = \begin{bmatrix} 3.0 \\ 4.0 \end{bmatrix}$ with parameters $\theta = \begin{bmatrix} 0.5 & -0.5 & 0.5 & -0.5 & 1 & -1 & -2 & 1 & 0.5 \end{bmatrix}^\top$.

Example: multilayer perceptron (MLP)

Exercise

In the MLP example, find the output for the input $x = \begin{bmatrix} 3.0 \\ 4.0 \end{bmatrix}$ with parameters $\theta = \begin{bmatrix} 0.5 & -0.5 & 0.5 & -0.5 & 1 & -1 & -2 & 1 & 0.5 \end{bmatrix}^\top$.

Solution:

- **Node 4 (Hidden 1):** Sum is $(0.5 \times 3) + (-0.5 \times 4) + (1 \times 1) = 0.5$.
 $r^{(4)} = [\text{ReLU}(0.5)] = [0.5]$.
- **Node 5 (Hidden 2):** Sum is $(0.5 \times 3) + (-0.5 \times 4) + (-1 \times 1) = -1.5$.
 $r^{(5)} = [\text{ReLU}(-1.5)] = [0.0]$.
- **Node 6 (Output):** Sum is $(-2 \times 0.5) + (1 \times 0.0) + (0.5 \times 1) = -0.5$.
 $r^{(6)} = [-0.5]$.

Final Output: $y = [-0.5]$.

Example: 1D convolution

As an example of **parameter sharing**, let's consider a 1D convolutional layer with a kernel of size 2, $\mathbf{k} = [k_1, k_2]^\top$.

$$y_1 = k_1 x_1 + k_2 x_2$$

$$y_2 = k_1 x_2 + k_2 x_3$$

$$y_3 = k_1 x_3 + k_2 x_4$$

The same parameters k_1, k_2 are reused.

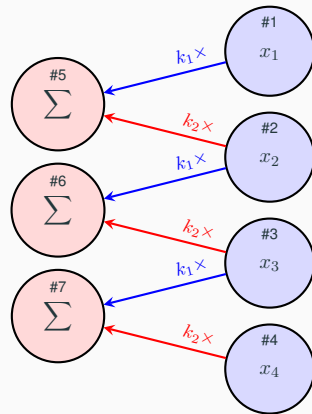


Figure 9: DAG of a 1D convolution. Edges of the same color share the same parameter.

Example: 1D convolution

Let's execute the computation.

- Checkpoint: $\theta = [k_1, k_2]^\top = [0.5, -1.0]^\top$.
- Input: $x = [2.0, -1.0, 3.0, 0.0]^\top$.

Example: 1D convolution

Let's execute the computation.

- Checkpoint: $\theta = [k_1, k_2]^\top = [0.5, -1.0]^\top$.
- Input: $x = [2.0, -1.0, 3.0, 0.0]^\top$.

1. **Nodes 1-4:** Return input values $[2.0], [-1.0], [3.0], [0.0]$.

2. **Node 5 (Output 1):**

- Arguments: $(\theta_1 \times r_1^{(1)}), (\theta_2 \times r_1^{(2)}) = (0.5 \times 2.0), (-1.0 \times -1.0)$.
- Sum: $1.0 + 1.0 = 2.0$. Return value: $r^{(5)} = [2.0]$.

Example: 1D convolution

Let's execute the computation.

- Checkpoint: $\theta = [k_1, k_2]^\top = [0.5, -1.0]^\top$.
- Input: $x = [2.0, -1.0, 3.0, 0.0]^\top$.

1. **Nodes 1-4:** Return input values $[2.0], [-1.0], [3.0], [0.0]$.

2. **Node 5 (Output 1):**

- Arguments: $(\theta_1 \times r_1^{(1)}), (\theta_2 \times r_1^{(2)}) = (0.5 \times 2.0), (-1.0 \times -1.0)$.
- Sum: $1.0 + 1.0 = 2.0$. Return value: $r^{(5)} = [2.0]$.

3. **Node 6 (Output 2):**

- Arguments: $(\theta_1 \times r_1^{(2)}), (\theta_2 \times r_1^{(3)}) = (0.5 \times -1.0), (-1.0 \times 3.0)$.
- Sum: $-0.5 - 3.0 = -3.5$. Return value: $r^{(6)} = [-3.5]$.

Example: 1D convolution

4. Node 7 (Output 3):

- Arguments: $(\theta_1 \times r_1^{(3)}), (\theta_2 \times r_1^{(4)}) = (0.5 \times 3.0), (-1.0 \times 0.0)$.
- Sum: $1.5 + 0.0 = 1.5$. Return value: $r^{(7)} = [1.5]$.

Example: 1D convolution

4. Node 7 (Output 3):

- Arguments: $(\theta_1 \times r_1^{(3)}), (\theta_2 \times r_1^{(4)}) = (0.5 \times 3.0), (-1.0 \times 0.0)$.
- Sum: $1.5 + 0.0 = 1.5$. Return value: $r^{(7)} = [1.5]$.

5. **Network Output:** The output is the concatenation of the return values from nodes 5, 6, and 7.

$$\mathbf{y} = \begin{bmatrix} r_1^{(5)} \\ r_1^{(6)} \\ r_1^{(7)} \end{bmatrix} = \begin{bmatrix} 2.0 \\ -3.5 \\ 1.5 \end{bmatrix}$$

Example: 1D convolution

Remark (Definition of Convolution)

The convolution operation, denoted by “ $\ast$ ”, is more generally defined by considering **padding** (adding zeros to the input ends) and **stride** (the step size when sliding the kernel).

The output size d_{out} depends on the input size d_{in} , kernel size d_{k} , padding p , and stride s :

$$d_{\text{out}} = \lfloor \frac{d_{\text{in}} + 2p - d_{\text{k}}}{s} \rfloor + 1$$

Our example was a case of $p = 0$ (no padding) and $s = 1$ (stride 1).

Example: 1D convolution

Exercise

In the 1D convolution example, find the output for the input $x = \begin{bmatrix} 2.0 \\ -1.0 \\ 3.0 \\ 0.0 \end{bmatrix}$ with parameters $\theta = [k_1, k_2]^\top = \begin{bmatrix} 1.5 \\ 1.0 \end{bmatrix}$.

Example: 1D convolution

Exercise

In the 1D convolution example, find the output for the input $x = \begin{bmatrix} 2.0 \\ -1.0 \\ 3.0 \\ 0.0 \end{bmatrix}$ with parameters $\theta = [k_1, k_2]^\top = \begin{bmatrix} 1.5 \\ 1.0 \end{bmatrix}$.

Solution:

- Output 1 (Node 5): $(1.5 \times 2.0) + (1.0 \times -1.0) = 3.0 - 1.0 = 2.0$.
- Output 2 (Node 6): $(1.5 \times -1.0) + (1.0 \times 3.0) = -1.5 + 3.0 = 1.5$.
- Output 3 (Node 7): $(1.5 \times 3.0) + (1.0 \times 0.0) = 4.5 + 0.0 = 4.5$.

Final Output: $y = [2.0, 1.5, 4.5]^\top$.

Example: a complex network

Our framework allows for irregular structures, multi-dimensional outputs from nodes, and multiple edges between nodes.

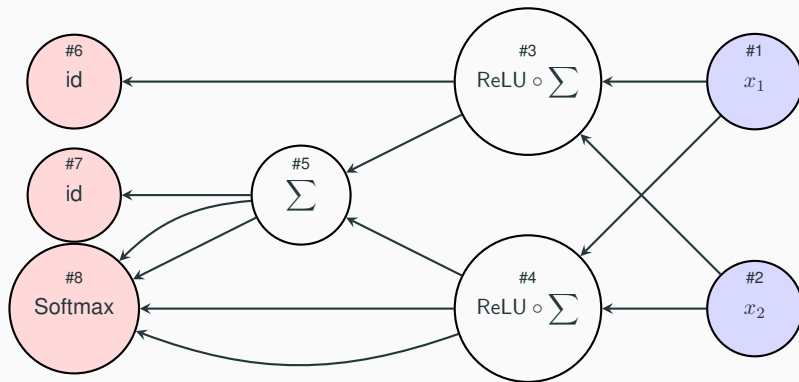


Figure 10: Example of a network with a complex structure

Example: a complex network

Input: $x = [1.0, -2.0]^\top$. Weights are given.

1. **Nodes 1, 2:** $r^{(1)} = [1.0]$, $r^{(2)} = [-2.0]$.

Example: a complex network

Input: $x = [1.0, -2.0]^\top$. Weights are given.

1. **Nodes 1, 2:** $r^{(1)} = [1.0]$, $r^{(2)} = [-2.0]$.
2. **Node 3:** ($\theta_{(1,3)} = 1, \theta_{(2,3)} = 1$)
 - Sum of args: $(1 \times 1.0) + (1 \times -2.0) = -1.0$.
 - Return value: $r^{(3)} = [\text{ReLU}(-1.0)] = [0.0]$.

Example: a complex network

Input: $x = [1.0, -2.0]^\top$. Weights are given.

1. **Nodes 1, 2:** $r^{(1)} = [1.0]$, $r^{(2)} = [-2.0]$.
2. **Node 3:** ($\theta_{(1,3)} = 1, \theta_{(2,3)} = 1$)
 - Sum of args: $(1 \times 1.0) + (1 \times -2.0) = -1.0$.
 - Return value: $r^{(3)} = [\text{ReLU}(-1.0)] = [0.0]$.
3. **Node 4:** ($\theta_{(1,4)} = 1, \theta_{(2,4)} = -1$)
 - Sum of args: $(1 \times 1.0) + (-1 \times -2.0) = 3.0$.
 - Return value: $r^{(4)} = [\text{ReLU}(3.0)] = [3.0]$.

Example: a complex network

4. **Node 5:** $(\theta_{(3,5)} = 0.5, \theta_{(4,5)} = -0.5)$

- Sum of args: $(0.5 \times r_1^{(3)}) + (-0.5 \times r_1^{(4)}) = (0.5 \times 0.0) + (-0.5 \times 3.0) = -1.5$.
- Return value: $\mathbf{r}^{(5)} = [-1.5]$.

Example: a complex network

4. **Node 5:** ($\theta_{(3,5)} = 0.5, \theta_{(4,5)} = -0.5$)

- Sum of args: $(0.5 \times r_1^{(3)}) + (-0.5 \times r_1^{(4)}) = (0.5 \times 0.0) + (-0.5 \times 3.0) = -1.5$.
- Return value: $r^{(5)} = [-1.5]$.

5. **Node 6:** ($\theta_{(3,6)} = 1$)

- Argument: $(1 \times r_1^{(3)}) = 0.0$. Return value: $r^{(6)} = [0.0]$.

Example: a complex network

4. **Node 5:** ($\theta_{(3,5)} = 0.5, \theta_{(4,5)} = -0.5$)

- Sum of args: $(0.5 \times r_1^{(3)}) + (-0.5 \times r_1^{(4)}) = (0.5 \times 0.0) + (-0.5 \times 3.0) = -1.5$.
- Return value: $r^{(5)} = [-1.5]$.

5. **Node 6:** ($\theta_{(3,6)} = 1$)

- Argument: $(1 \times r_1^{(3)}) = 0.0$. Return value: $r^{(6)} = [0.0]$.

6. **Node 7:** ($\theta_{(5,7)} = 2$)

- Argument: $(2 \times r_1^{(5)}) = -3.0$. Return value: $r^{(7)} = [-3.0]$.

Example: a complex network

8. **Node 8:** This node has 4 arguments coming from nodes 4 and 5.

- $a_1^{(8)} = \theta_{(4,8),1} r_1^{(4)} = 1 \times 3.0 = 3.0$
- $a_2^{(8)} = \theta_{(4,8),2} r_1^{(4)} = -1 \times 3.0 = -3.0$
- $a_3^{(8)} = \theta_{(5,8),1} r_1^{(5)} = 1 \times -1.5 = -1.5$
- $a_4^{(8)} = \theta_{(5,8),2} r_1^{(5)} = 1 \times -1.5 = -1.5$

Example: a complex network

8. **Node 8:** This node has 4 arguments coming from nodes 4 and 5.

- $a_1^{(8)} = \theta_{(4,8),1} r_1^{(4)} = 1 \times 3.0 = 3.0$
- $a_2^{(8)} = \theta_{(4,8),2} r_1^{(4)} = -1 \times 3.0 = -3.0$
- $a_3^{(8)} = \theta_{(5,8),1} r_1^{(5)} = 1 \times -1.5 = -1.5$
- $a_4^{(8)} = \theta_{(5,8),2} r_1^{(5)} = 1 \times -1.5 = -1.5$

The activation function computes $\text{Softmax} \left(\begin{bmatrix} a_1 + a_3 \\ a_2 + a_4 \end{bmatrix} \right)$.

$$z = \begin{bmatrix} 3.0 - 1.5 \\ -3.0 - 1.5 \end{bmatrix} = \begin{bmatrix} 1.5 \\ -4.5 \end{bmatrix}$$

$$r^{(8)} = \text{Softmax}(z) = \frac{1}{e^{1.5} + e^{-4.5}} \begin{bmatrix} e^{1.5} \\ e^{-4.5} \end{bmatrix} \approx \begin{bmatrix} 0.9975 \\ 0.0025 \end{bmatrix}$$

Example: a complex network

8. **Node 8:** This node has 4 arguments coming from nodes 4 and 5.

- $a_1^{(8)} = \theta_{(4,8),1} r_1^{(4)} = 1 \times 3.0 = 3.0$
- $a_2^{(8)} = \theta_{(4,8),2} r_1^{(4)} = -1 \times 3.0 = -3.0$
- $a_3^{(8)} = \theta_{(5,8),1} r_1^{(5)} = 1 \times -1.5 = -1.5$
- $a_4^{(8)} = \theta_{(5,8),2} r_1^{(5)} = 1 \times -1.5 = -1.5$

The activation function computes $\text{Softmax} \left(\begin{bmatrix} a_1 + a_3 \\ a_2 + a_4 \end{bmatrix} \right)$.

$$z = \begin{bmatrix} 3.0 - 1.5 \\ -3.0 - 1.5 \end{bmatrix} = \begin{bmatrix} 1.5 \\ -4.5 \end{bmatrix}$$

$$r^{(8)} = \text{Softmax}(z) = \frac{1}{e^{1.5} + e^{-4.5}} \begin{bmatrix} e^{1.5} \\ e^{-4.5} \end{bmatrix} \approx \begin{bmatrix} 0.9975 \\ 0.0025 \end{bmatrix}$$

Final Output: Concatenating outputs from nodes 6, 7, and 8 gives

$$y \approx [0.0, -3.0, 0.9975, 0.0025]^\top.$$

Example: a complex network

Exercise

For the same complex architecture and input $x = \begin{bmatrix} 1.0 \\ -2.0 \end{bmatrix}$, calculate the output with a different set of parameters (provided in the lecture notes).

Example: a complex network

Exercise

For the same complex architecture and input $x = \begin{bmatrix} 1.0 \\ -2.0 \end{bmatrix}$, calculate the output with a different set of parameters (provided in the lecture notes).

Solution:

- Node 3 output $r^{(3)} = [\text{ReLU}(1.5)] = [1.5]$.
- Node 4 output $r^{(4)} = [\text{ReLU}(1.0)] = [1.0]$.
- Node 5 output $r^{(5)} = [1.5 + 1.0] = [2.5]$.
- Node 6 output $r^{(6)} = [-2 \times 1.5] = [-3.0]$.
- Node 7 output $r^{(7)} = [1 \times 2.5] = [2.5]$.
- Node 8 input vector $z = \begin{bmatrix} (0.5 \times 1.0) + (-1 \times 2.5) \\ (0.5 \times 1.0) + (1 \times 2.5) \end{bmatrix} = \begin{bmatrix} -2.0 \\ 3.0 \end{bmatrix}$.
- Node 8 output $r^{(8)} = \text{Softmax}(\begin{bmatrix} -2.0 \\ 3.0 \end{bmatrix}) \approx [0.0067, 0.9933]^\top$.

Final Output: $y \approx [-3.0, 2.5, 0.0067, 0.9933]^\top$.

DAG-based definition's generality - answer

Question: Does the definition really include practical neural networks?

Answer: Yes, it includes many practical neural networks. (Not difficult to confirm that it also includes RNNs and transformers.)

Outline

1. Introduction
2. Preparation: Mathematical Notations
3. What does “neural network” truly refer to? - DAG-based parametric function
- 4. Does the DAG-based definition include practical neural networks?**
5. Architecture and Checkpoints
6. Summary and Future Outlook

Outline

1. Introduction
2. Preparation: Mathematical Notations
3. What does “neural network” truly refer to? - DAG-based parametric function
4. Does the DAG-based definition include practical neural networks?

5. Architecture and Checkpoints

6. Summary and Future Outlook

Question: When we go to generative AI repository, we often see the words “architecture” or “checkpoint.” What do they formally mean, according to our definitions?

Architecture and checkpoints

Let's summarize the key components of a neural network based on our discussion.

1. A human first designs the computational blueprint: the DAG, activation functions, and parameter assignments.
2. Then, a learning process (**training**) finds the specific numerical values for the parameters suitable for the task.

Architecture and checkpoints

Let's summarize the key components of a neural network based on our discussion.

1. A human first designs the computational blueprint: the DAG, activation functions, and parameter assignments.
2. Then, a learning process (**training**) finds the specific numerical values for the parameters suitable for the task.

Definition (Architecture and Checkpoint)

- **Architecture:** The design information of a neural network, i.e., **everything other than the specific values of the parameters**. (This is what we formally defined earlier).
- **Checkpoint:** A set of **specific parameter values** (a list of numbers) that have been learned for a specific architecture.

Architecture and checkpoints

Remark (The Ambiguity of the Word "Model")

The term "**model** of a neural network" can be ambiguous, so caution is required.

- In theoretical contexts or research papers, "model" often refers **only to the architecture**. (e.g., "The Transformer model").

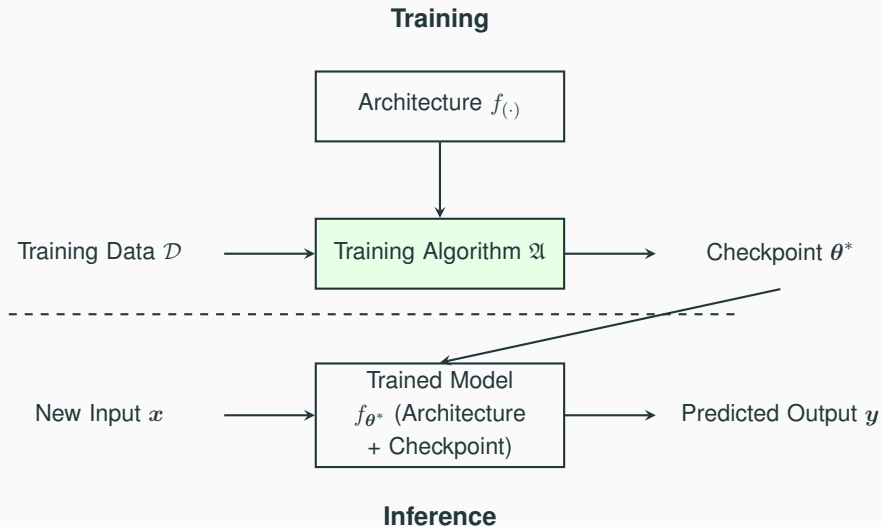
Architecture and checkpoints

Remark (The Ambiguity of the Word "Model")

The term "**model** of a neural network" can be ambiguous, so caution is required.

- In theoretical contexts or research papers, "model" often refers **only to the architecture**. (e.g., "The Transformer model").
- In applied contexts or software libraries, "model" often refers to the **pair of an architecture and a checkpoint**. (e.g., "a pre-trained model").

Architecture and checkpoints



Architecture and checkpoint - answer

Question: When we go to generative AI repository, we often see the words “architecture” or “checkpoint.” What do they formally mean, according to our definitions?

Answer:

Architecture and checkpoint - answer

Question: When we go to generative AI repository, we often see the words “architecture” or “checkpoint.” What do they formally mean, according to our definitions?

Answer:

- The architecture $f_{(\cdot)}$ is the structure information of a neural network.
- The checkpoint θ is specific parameter values.

Outline

1. Introduction
2. Preparation: Mathematical Notations
3. What does “neural network” truly refer to? - DAG-based parametric function
4. Does the DAG-based definition include practical neural networks?

5. Architecture and Checkpoints

6. Summary and Future Outlook

Outline

1. Introduction
2. Preparation: Mathematical Notations
3. What does “neural network” truly refer to? - DAG-based parametric function
4. Does the DAG-based definition include practical neural networks?
5. Architecture and Checkpoints
- 6. Summary and Future Outlook**

Today's summary

Today's summary

- A neural network is a parametric function defined by a DAG and the activation functions associated with its nodes and parameter associated with its edges.

Today's summary

- A neural network is a parametric function defined by a DAG and the activation functions associated with its nodes and parameter associated with its edges.
- The output of the neural network is iteratively determined in the direction of the directed graph structure.

Today's summary

- A neural network is a parametric function defined by a DAG and the activation functions associated with its nodes and parameter associated with its edges.
- The output of the neural network is iteratively determined in the direction of the directed graph structure.
- For a neural network to determine a function, we need the two components: the **architecture** (DAG, activation functions, edge-parameter correspondence) and the **checkpoint** (the specific parameter values).

Preview of the next lecture

This time, we have seen that a neural network is a parametric function composed of two elements: the architecture and the checkpoint. With an understanding of this distinction, you are ready to use neural networks as black boxes.

Preview of the next lecture

This time, we have seen that a neural network is a parametric function composed of two elements: the architecture and the checkpoint. With an understanding of this distinction, you are ready to use neural networks as black boxes.

Next time, we will discuss the inescapable issue of **licenses** when using publicly available neural network models.

Preview of the next lecture

This time, we have seen that a neural network is a parametric function composed of two elements: the architecture and the checkpoint. With an understanding of this distinction, you are ready to use neural networks as black boxes.

Next time, we will discuss the inescapable issue of **licenses** when using publicly available neural network models.

In fact, the distinction between architecture and checkpoint, and between "training" and "inference," which we learned today, is critically important for understanding the scope of license application.

Appendix: constraints on activation functions i

In theory, any function can be an activation function. In practice, we choose functions with specific properties for two main reasons:

1. **Possibility of Training:** We need to use **gradient methods** for efficient training. This requires the gradient (how a small parameter change affects the output) to be well-defined.
2. **Reusability of Parameters:** Techniques like **fine-tuning** (reusing a trained checkpoint for a new task) require stability. A small change in parameters should only cause a small change in the output.

Appendix: constraints on activation functions ii

Appendix: constraints on activation functions iii

- It mathematically justifies the use of gradient methods.
- It ensures the entire neural network is also locally Lipschitz continuous with respect to its parameters θ , enabling stable training and fine-tuning.